CS – 432: Databases

Assignment – I

Multi Course Attendance Management Portal

The GitHub Link to access the project content is given below:

https://github.com/Pallav-Biyala/Attendance_Management_Portal

Introduction to the Project:

In many colleges and schools, attendance is one of the important part of the student as well as teacher's life. Attendance ensures academic discipline of students as well as institutional accountability. Traditional attendance systems, which rely on manual recording through registers or fragmented digital solutions are always prone to error, inconsistencies and lack of transparency. As the number of students, courses and academic activities increase, maintaining accurate and structured attendance records becomes more challenging.

So to conquer this challenge, our team has decided to use the concept of databases and create a robust relational database system for managing academic attendance in university environment.

Team Members:

The members of this team who are working on this database are:

Sr. No.	Name	Roll Number
1.	Aryan Ashwin Meshram	24110054
2.	Jahan Zaib	24110140
3.	Mateen Sharief	24110197
4.	Pallav Biyala	24100234
5.	Saksham Chourasia	24110312

Objective of Project:

The objective of this project is to design and implement a robust relational database system for handling and properly managing academic Attendance for students of all years across various number of courses. This system aims to provide a structured and normalized framework to handle:

- Student enrollment in multiple courses
- Lecture logging
- Attendance Recording
- Attendance Correction
- Administrative Monitoring

→ Expected Core Functionalities of Attendance Management Database:

The Attendance Management Database is designed to be able to handle attendance recording within a university environment. The core functionalities that we added in our database are listed below:

1. **Student and Administrative Management:** The system contains a centralized **member entity** that represent all users including students, admin as well as teachers.
   The purpose to add this was:
   - Students can be uniquely identified and linked to their academic department trough their member ID
   - Administrators being responsible for monitoring attendance records, approving requests, and maintaining system integrity

   This role-based structure ensures controlled access and accountability.

2. **Course and Slot Management:** Attendance depends on the number of courses one has registered and hence it is very important to have database for courses too. Courses are associated with academic departments and are assigned specific time slots through the **course slot** entity.
    - Each course may have multiple slots.
    - Slot information ensures proper lecture scheduling so that there is no clash between mandatory courses for a department
    - Courses are uniquely identified to prevent duplication.

3. **Enrollment Management:** Since students can take multiple courses and courses can have multiple departments, it was necessary to keep track of courses a student registered. Hence for that, our database provides **Enrollment Entity.**
    - A student may enroll in multiple courses. A course may have multiple students.
    - Hence, Composite keys are kept ensure that duplicate enrollments are prevented.

    Enrollment validation is enforced before attendance can be recorded.

4. **Lecture Logging:** It is very necessary to keep track of each and every lecture that is planned as per course slot and is actually conducted or not. This is because students must be given attendance of only those lectures where it was conducted. If class was not conducted, then they must not be marked absent in that. This all is make sure by our **Lecture Log entity.**

    It includes all the lectures that are planned and in case an extra class was taken, it can include that too. The isconducted column makes sure students are marked present/absent only for those classes which are conducted.

    Lecture logging provides structured tracking of academic sessions.

5. **Attendance Recording:** All the attendance of lectures conducted is stored in the attendance table for each student for all courses they took.
    - Attendance entries are linked to both the student and the corresponding lecture.
    - Composite foreign key constraints ensure that attendance can only be recorded for enrolled students.
    - Duplicate attendance entries are prevented.

    This design ensures referential integrity and prevents invalid records.

6. **Attendance Request:** Our database's another functionality is to allow students to send their queries if they are marked wrongly in attendance. This is handled by **attendance_request entity.** Students may submit attendance correction requests through the attendance_request table.
    - Requests are linked to specific attendance records.
    - Administrators can approve or reject requests.
    - The system maintains request status tracking.

    This functionality introduces a controlled mechanism for attendance modification.

7. **Notification System:** This is the latest feature that our database handles. We have created notification table which with help of **attendance_report** keeps track of percentage of students for each course. The course entity contains percentage threshold. If a student enrolled in that reaches attendance below that threshold, the students will be notified that their attendance percentage is below threshold.
    This is done by our **Notification entity.**

    The notification table enables communication between administrators and students.
    - This alerts students if their attendance is below threshold as set by course in course entity
    - It also tells students if their request (in attendance_request) is approved or rejected by faculty or not

Hence, this feature ensures transparency and user awareness

8. **Audit Logging:** To ensure accountability, our database system maintains **audit_log entity.**
    - Critical modifications are recorded.
    - Administrator actions are tracked.
    - Historical changes are preserved.

    Audit logging strengthens data governance and prevents unauthorized modifications.

Hence, in short our system fully supports attendance reporting through structured queries and database views.

- Attendance percentage calculations can be derived through queries
- Reports can be generated per student or per course.
- The design allows efficient data retrieval for analysis
- Also, if as student wants to know how many classes he/she can miss, it is possible to know that using the threshold, the total number of classes for each course from lecture log and how many he/she attended. Hence students can also get idea how many classes they can miss without losing marks

Tables in our Database:

There are in a total of 14 tables in our database:

1. Member
2. Student
3. Teacher
4. Admin
5. Department
6. TeacherDepartment
7. Course
8. CourseSlot
9. Enrollment
10. LectureLog
11. Attendance
12. Attendance_request
13. Notification
14. Audit_log

NOTE:

In our database schema, there appears to be a fifteenth entity named attendance_report. However, it is important to clarify that attendance_report is implemented as a **database view**, not as a physical table.

This view is designed to dynamically calculate and display the attendance percentage of students for their respective enrolled courses.

The attendance percentage is a derived attribute computed using data from the attendance, lecturelog, and enrollment tables. Since it is a calculated value, storing it in a separate physical table would **introduce redundancy** and potentially **violate normalization principles**.

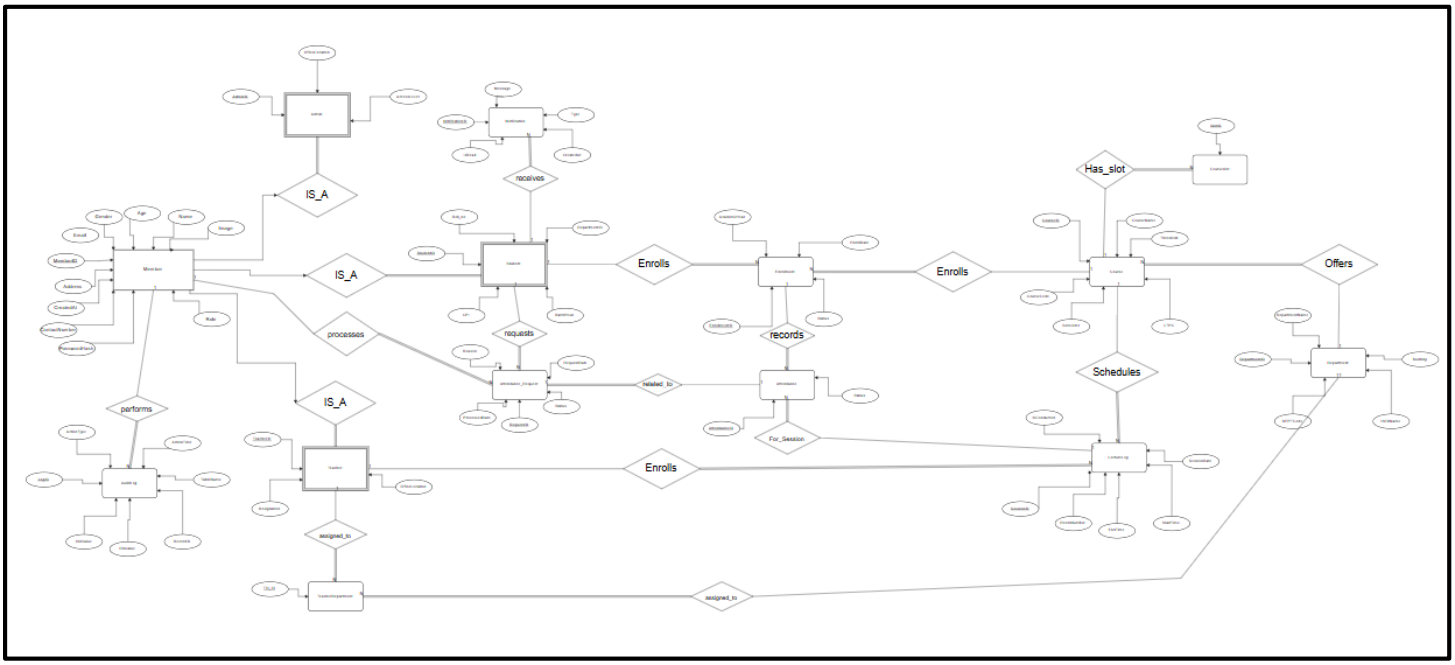
The attendance_report view is still created for the following purposes:

- On-demand calculation of attendance percentage
- Real-time reflection of updates made to attendance records
- Simplified querying for reporting and analysis
- To generate notifications easily

It is important to emphasize that **attendance_report does not store independent data**. Instead, it retrieves and aggregates information from existing base tables. Therefore, **it should not be considered a base table** in the relational schema but rather a derived reporting component of the system.

Module – B: ER Modelling

ER Diagram:



So this was the ER diagram for the whole attendance management database. It shows how the entities interact, the cardinalities involved, and any associative entities used to resolve many-to-many relationships.

Note: For better view, the ER Diagram is also provided in the Github link given below:

https://github.com/Pallav-Biyala/Attendance_Management_Portal

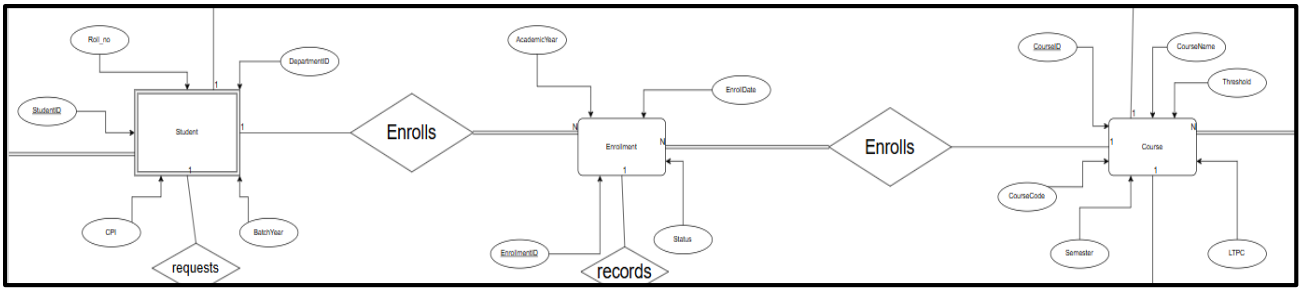
Relationships and Their Justification:

1. **Student — Enrols — Course**
   Justification:
   - A Student can enrol in multiple Courses.
   - A Course can have multiple Members enrolled.
   - M:N relationship.
   - Hence, to normalize it we added Enrollment Table, that's why we can see 1:N / N:1 etc because of normalization.

   To store additional attributes such as:
   - EnrollmentDate
   - Status

   Enrolment is modelled as an associative entity to properly represent the many-to-many relationship and to store enrolment-specific attributes.

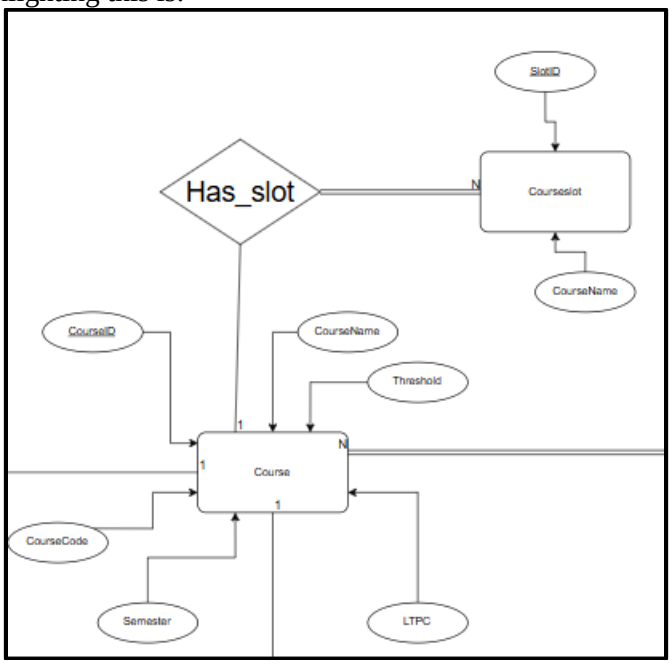
   The part of ER Diagram highlighting this is:



2. **Course — Has_slot — CourseSlot**
   Justification:
   - A Course may have multiple time slots.
   - Each CourseSlot belongs to exactly one Course.
   - 1:N relationship.

   This ensures structured scheduling and prevents slots from existing independently without a course.

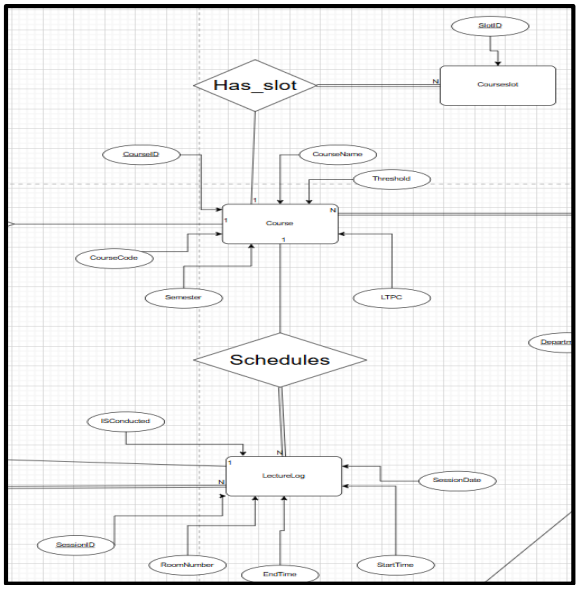
   The part of ER Diagram highlighting this is:



3. **CourseSlot — For_Session — LectureLog**
   Justification:
   - Each CourseSlot can generate multiple lecture sessions over time.
   - Each LectureLog corresponds to exactly one CourseSlot.
   - 1:N relationship.

   This separation ensures that recurring slots and actual conducted lectures are distinguished.

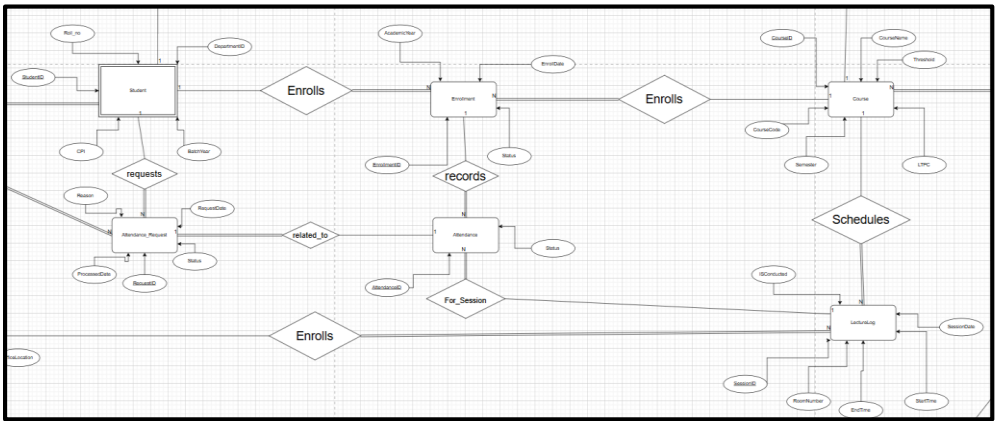
   The part of ER Diagram highlighting this is:



4. **Member — Attendance — LectureLog**
   Justification:
   - A Member can attend multiple LectureLogs.
   - Each LectureLog records attendance for multiple Members.
   - M:N relationship.
   - Hence to normalize it we added Attendance table, that's why we can see 1:N / N:1 etc because of normalization.

   Attendance is modelled as a separate entity to:
   - Record status (Present/Absent)
   - Maintain session-specific tracking
   - Enable analytics

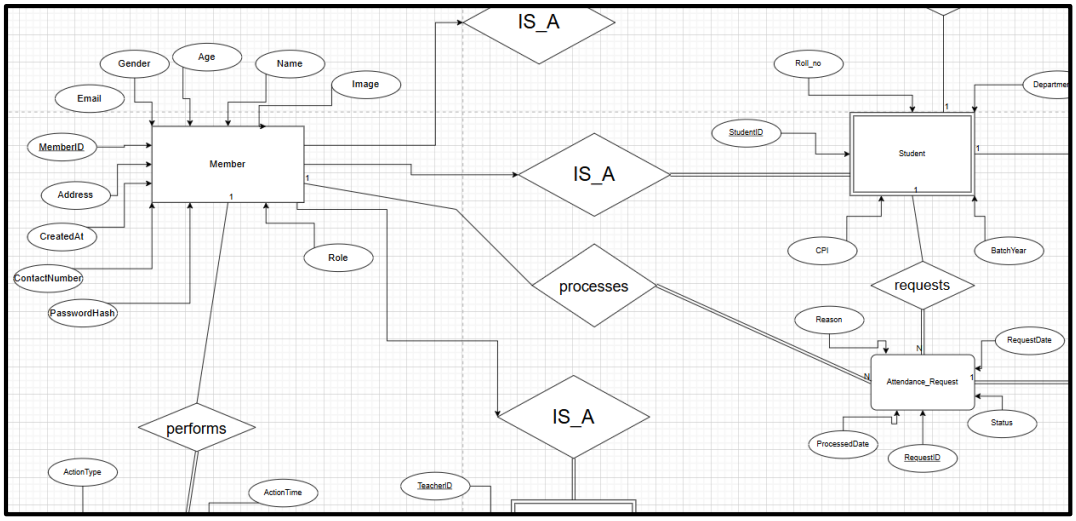
   The part of ER Diagram highlighting this is:



5. **Member — Attendance_Request**
   Justification:
   - A Member can raise multiple attendance requests.
   - Each Attendance_Request belongs to exactly one Member.
   - 1:N relationship.

   This reflects real-world workflow where requests originate from individual members.

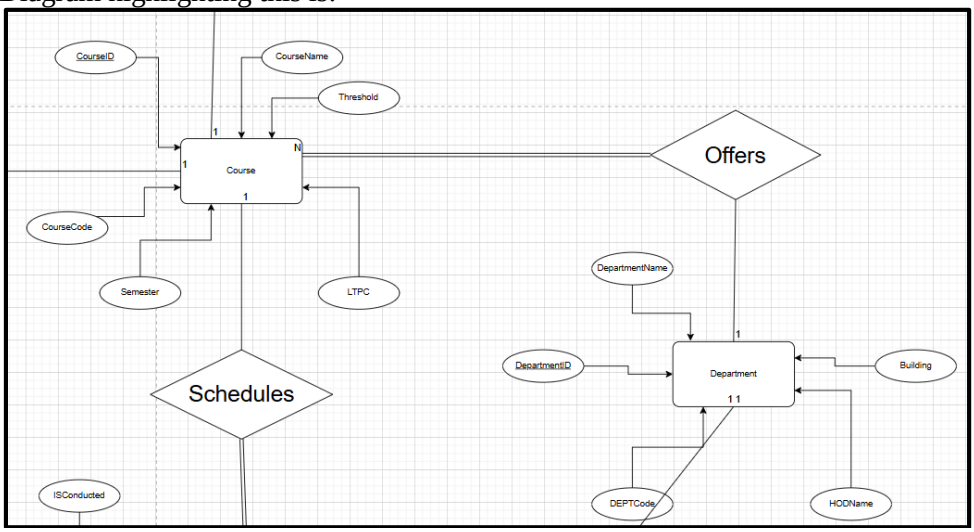
   The part of ER Diagram highlighting this is:



6. **Department — Offers — Course**
   Justification:
   - A Department can offer multiple Courses.
   - Each Course belongs to one Department.
   - Hence, **1:N relationship**.

   This enforces academic structure and hierarchy.

   The part of ER Diagram highlighting this is:



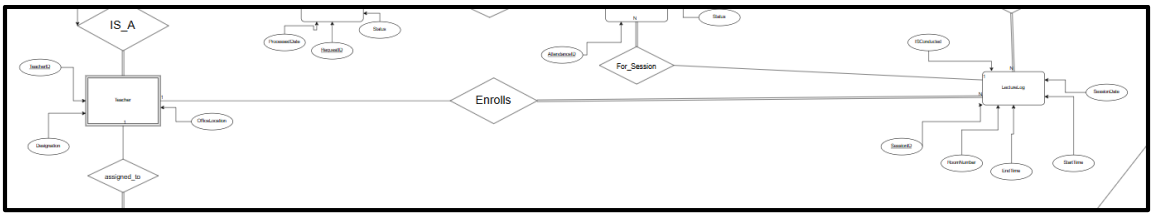
7. **IS_A (Member Specialization)**
   Justification:
   Member is specialized into role-based subtypes such as:
   - Student
   - Instructor (teacher)
   - Admin

   This allows:
   - Role-specific attributes
   - Controlled access levels
   - Clean role separation

8. **Teacher – Course:**
   Justification
   - Teacher can teach multiple Courses.

- Each Course can have multiple Teachers.
- Allows assigning co-teaching or multiple instructors per course in lecture log
- M:N Relationship
- Hence, that's another reason we added Lecture Log table to normalize it and that's why we can see 1:N / N:1 etc because of normalization.

The part of ER Diagram highlighting this is:

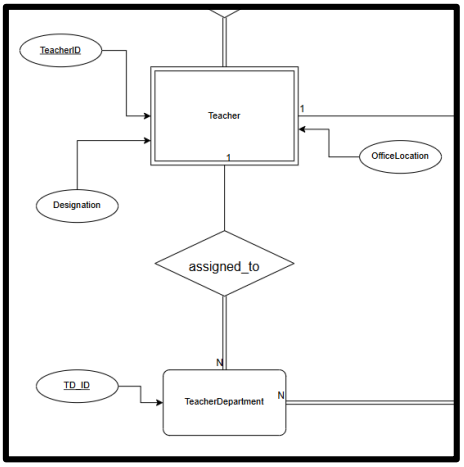


9. Teacher – Department:

Justification:

- Teacher can be assigned to multiple TeacherDepartments.
- Department can have multiple teachers.
- Maintains clear structure for departmental responsibilities.
- M:N relationship
- Hence to normalize it, we added teacherdepartment table, that's why we can see 1:N / N:1 etc because of normalization.

The part of ER diagram highlighting this is:

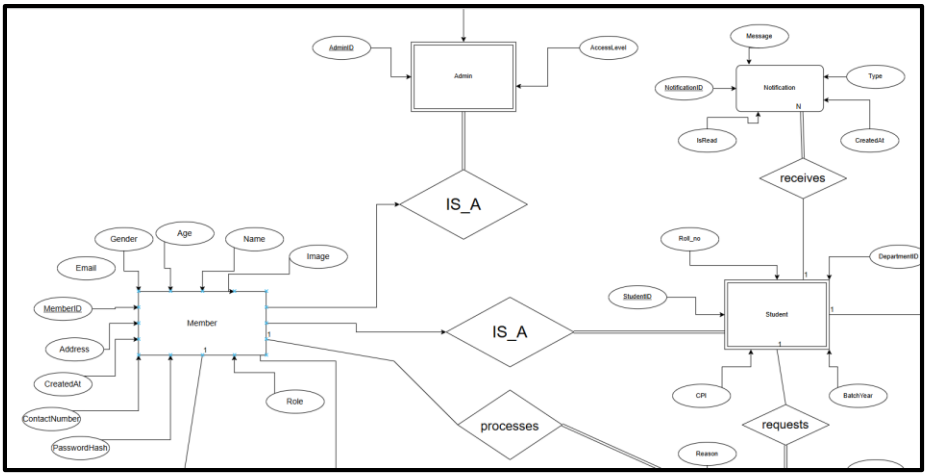


10. **Member — Receives — Notification**

Justification:

- A Member can receive multiple notifications.
- Each Notification is sent to exactly one Member.
- Enables alerting students and teachers about updates or requests.
- 1:N Relationship

The part of ER Diagram highlighting this is:



Additional Constraints:

1. **Cardinality Constraints:**

   - A Member may enroll in multiple Courses.
   - A Course must have zero or more enrolled Members.
   - Each Attendance record must correspond to exactly one LectureLog and one Member.
   - Each Attendance_Request must belong to exactly one Member.

2. **Participation Constraints:**

   - Attendance cannot exist without both Member and LectureLog.
   - Attendance_Request cannot exist without a Member.
   - AuditLog cannot exist without an Admin.
   - To enforce logical dependency.

3. **Uniqueness Constraints:**

   - MemberID is unique.
   - CourseID is unique.
   - DepartmentID is unique.
   - EnrollmentID uniquely identifies each enrollment.
   - AttendanceID uniquely identifies each attendance entry.

4. **Temporal Constraints:**

   - LectureLog must have valid StartTime and EndTime.
   - Attendance marking is valid only after lecture is conducted.
   - EnrollmentDate must precede attendance marking.

5. **Business Rules:**
   - A Member cannot mark attendance without enrollment.
   - An Attendance_Request must refer to an existing attendance record.
   - Only Admin can process requests.
   - AuditLog must record every administrative action.

6. **Referential Integrity:**
   - Deleting a Course should not orphan Attendance records.
   - Deleting a Member should cascade to Enrollment and Attendance.

- Department deletion should be restricted if Courses exist.

Team Members Contributions:

Sr. No.	Name	Roll No.	Justification
1.	Aryan Ashwin Meshram	24110054	Led the conceptual modelling phase with primary focus on designing and refining the ER diagrams, ensuring correct representation of entities, relationships, cardinalities, and constraints in alignment with database theory.
2.	Jahan Zaib	24110140	Contributed to UML modelling and assisted in mapping UML class diagrams to ER structures, ensuring conceptual-to-logical consistency.
3.	Mateen Sharief	24110197	Assisted in functional requirement analysis, query validation, testing referential integrity, and verification of logical constraints across implemented modules.
4.	Pallav Biyala	24110234	Took primary responsibility for data synthesis and SQL dump creation, including schema implementation, table creation, constraint enforcement (PK, FK, NOT NULL), and generating meaningful synthetic datasets.
5.	Saksham Chourasia	24110321	Coordinated overall schema design and system integration, contributed to relationship justification, normalization checks, constraint validation, and documentation preparation.